

higher, architectural level.

Consider, for example, an architect working on a system, S. He wants to include a certain framework, F, into the system. Now suppose that the authors of F have bound it to a particular database, D. So S depends on F, which depends on D ([Figure 10.3](#)).



Figure 10.3 A problematic architecture

Now suppose that D contains features that F does not use and, therefore, that S does not care about. Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S. Even worse, a failure of one of the features within D may cause failures in F and S.

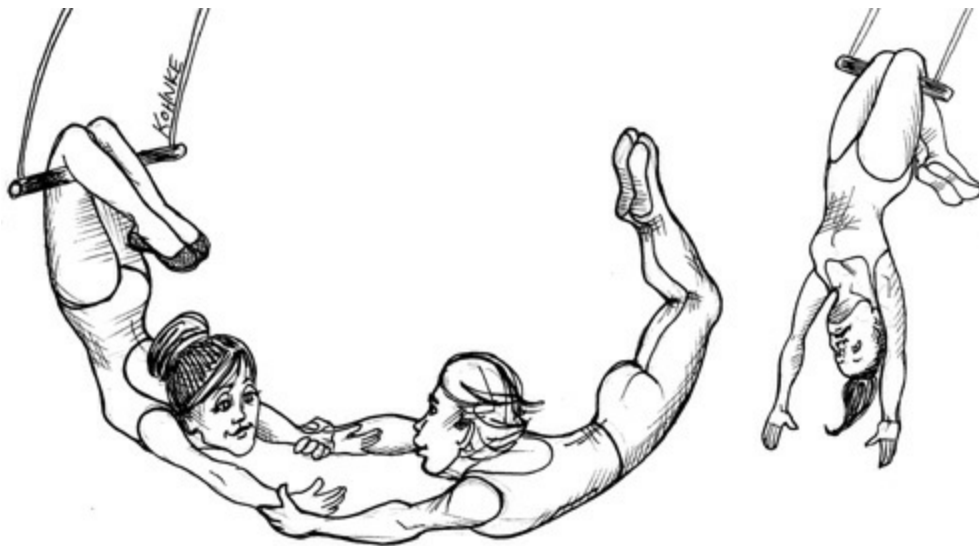
CONCLUSION

The lesson here is that depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.

We'll explore this idea in more detail when we discuss the Common Reuse Principle in [Chapter 13](#), "Component Cohesion."

11

DIP: THE DEPENDENCY INVERSION PRINCIPLE



The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.

In a statically typed language, like Java, this means that the `use`, `import`, and `include` statements should refer only to source modules containing interfaces, abstract classes, or some other kind of abstract declaration. Nothing concrete should be depended on.

The same rule applies for dynamically typed languages, like Ruby and Python. Source code dependencies should not refer to concrete modules. However, in these